

DB A4

Laiba Fatima

22k-5195

Q1.

S1: R1(A), R2(A), R3(A), R4(A), W1(A), W2(A), W3(A), W4(A)

1. Conflict Serializability:

- ⊄ Conflicting operations involve the same data item (A) and include at least one write operation.
- ⊄ Conflicting pairs:
 - W1(A) vs. W2(A), W3(A), W4(A)
 - W2(A) vs. W3(A), W4(A)
 - W3(A) vs. W4(A)
- ⊄ The precedence graph shows cycles (e.g., $T1 \rightarrow T2$, $T2 \rightarrow T3$, etc.).
- ⊄ S1 is not conflict serializable due to the presence of cycles.

2. View Serializability:

- ⊄ Check for initial read, final write, and read-from relationships.
- ⊄ The final write of A differs across transactions, and read-from relationships conflict.
- ⊄ S1 is not view serializable.

S2: R1(A), R2(A), W3(A), W1(A)

1. Conflict Serializability:

- ⊄ Conflicting pairs:
 - R1(A) vs. W3(A), W1(A)
 - R2(A) vs. W3(A), W1(A)
 - W3(A) vs. W1(A)
- ⊄ The precedence graph shows a cycle (e.g., $T1 \rightarrow T3 \rightarrow T1$).
- ⊄ S2 is not conflict serializable.

2. View Serializability:

- ⊄ Final writes and read-from relationships conflict.
- ⊄ S2 is not view serializable.

S3: R1(A), R2(A), W1(A), W2(A), R1(B), R2(B), W1(B), W2(B)

1. Conflict Serializability:

- ⌘ Conflicting pairs:
 - $W1(A)$ vs. $W2(A)$
 - $W1(B)$ vs. $W2(B)$
- ⌘ No cycles in the precedence graph.
- ⌘ $S3$ is conflict serializable.

2. View Serializability:

- ⌘ Initial reads, final writes, and read-from relationships align with a serial schedule.
- ⌘ $S3$ is view serializable.

Q2.

T1 adds 100 to the balance (initially 500), and T2 subtracts 50. Operations interleave as follows:

1. Concurrency Issues:

- Lost Update: The write of T1 (600) is overwritten by T2 (450), causing the update of T1 to be lost.
- Consequences: Data inconsistency and incorrect results.

2. Final Balance:

- Final balance = 450 (T2's write overwrites T1's write).

3. Correct Serial Outcome:

- Serial execution of T1 followed by T2: $(500 + 100) - 50 = 550$.
- Serial execution of T2 followed by T1: $(500 - 50) + 100 = 550$.
- Difference: The concurrent result (450) differs from the correct serial result (550).

Q3.

S1:

T1: Read(X), Write(X), Read(Y) T2: Read(X), Write(X), Write(Y)

S2:

T1: Read(X), Write(X), Read(Y) T2: Read(X), Write(X), Write(Y)

1. Conflicting Operations:

- S1:
 - T1: Read(X) vs. T2: Write(X)
 - T2: Read(X) vs. T1: Write(X)

■ T2: Write(X) vs. T1: Write(X)

- S2:
- ■ Same conflicts as S1.

2. Conflict Equivalence:

- The order of conflicting operations is identical in S1 and S2.
- S1 and S2 are conflict equivalent.

3. Conflict Serializability:

- Since S1 and S2 are conflict equivalent, they are also conflict serializable.
- Serializable schedule: Execute T1 entirely before T2, or vice versa.

Q4.

Precedence Graph Analysis:

- Nodes: Transactions (e.g., T1, T2, T3).
- Edges: Conflicts (Read-Write, Write-Read, Write-Write).
- Check for cycles:
 - If no cycles exist, the schedule is conflict serializable.
 - If cycles exist, it is not conflict serializable.

Final Answer: Analyze the graph provided in the assignment to determine serializability. If needed, provide a step-by-step explanation based on the graph's structure.

Q5.

Operations executed on multiple processors may not follow a total order due to simultaneous execution.

Effect:

- Conflict serializability depends on the ordering of conflicting operations.
- If operations on data items can be fully ordered, conflict serializability remains achievable.
- If total ordering is not feasible, it becomes challenging to define conflict serializability.

Final Answer: While the scenario may complicate achieving conflict serializability, it does not inherently invalidate it, provided that operations on individual data items are correctly ordered.